

SOLID PRINCIPLES



Agile methodlar kullanılırken hatanın daha hızlı bulunması ve kodun daha efektif çalışması için sahiplenilmiş prensiplerdir. SOLID:

- **Değişiklikleri Kolaylaştırır**
- **Kodun Yeniden Kullanılmasını Sağlar**
- **Testi Kolaylaştırır**
- **Uyum İçinde Çalışır**

1. Single Responsibility

Tek Sorumluluk Prensipleri, bir sınıfın, modülün veya fonksiyonun değişmek için yalnızca tek bir nedene sahip olması gerektiğini ifade eder. Nesne yönelimli tasarımda (OOD) bu kural, her bileşenin sistem içerisinde **sadece spesifik bir sorumluluk** alanına odaklanmasını şart koşar.

Amacı ve İşlevi: Bileşenlerin karmaşıklığını (complexity) azaltmak ve sistem modülleri arasındaki bağımlılığı (coupling) minimum seviyeye indirmektir. Bir sınıf birden fazla sorumluluk üstlendiğinde, bu sorumluluklardan birinde yapılacak bir değişiklik diğerlerini de etkileme riski taşır. SRP sayesinde kodun okunabilirliği, test edilebilirliği ve sürdürülebilirliği (maintainability) garanti altına alınır.



```
class Animal {
  constructor(name, feedingType, soundMade) {
    this.name = name;
    this.feedingType = feedingType;
    this.soundMade = soundMade;
  }

  nomenclature() {
    console.log(`The name of the animal is ${this.name}`);
  }

  sound() {
    console.log(`${this.name} ${this.soundMade}s`);
  }

  feeding() {
    console.log(`${this.name} is a ${this.feedingType}`);
  }
}

let elephant = new Animal('Elephant', 'herbivore', 'trumpet');
elephant.nomenclature(); // The name of the animal is Elephant
elephant.sound(); // Elephant trumpets
elephant.feeding(); // Elephant is a herbivore
```

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  nomenclature() {
    console.log(`The name of the animal is ${this.name}`);
  }
}

let animal1 = new Animal('Elephant');
animal1.nomenclature(); // The name of the animal is Elephant

// Sound class
class Sound {
  constructor(name, soundMade) {
    this.name = name;
    this.soundMade = soundMade;
  }

  sound() {
    console.log(`${this.name} ${this.soundMade}s`);
  }
}

let animalSound1 = new Sound('Elephant', 'trumpet');
animalSound1.sound(); //Elephant trumpets

// Feeding class
class Feeding {
  constructor(name, feedingType) {
    this.name = name;
    this.feedingType = feedingType;
  }

  feeding() {
    console.log(`${this.name} is a/an ${this.feedingType}`);
  }
}

let animalFeeding1 = new Feeding('Elephant', 'herbivore');
animalFeeding1.feeding(); // Elephant is a/an herbivore
```

Bu iki koddan sağda olan Single Responsibility prensibine uygun olandır. Sağdaki kodda aynı class yapısı içinde hem isim hem beslenme türü hem de beslenme tipi verilmiştir. Bir yazılımcı bu class yapısında bir problemle karşılaştığı takdirde debug etmesi daha zor olacaktır.

2. Open-Closed Principle (OCP)

Açık/Kapalı Prensibi, yazılım varlıklarının (sınıflar, modüller, fonksiyonlar vb.) **gelişime** (uzantılara) **açık**, ancak **değişime kapalı** olması gerektiğini belirten kuraldır.

Amacı ve İşlevi: Sisteme yeni işlevler veya gereksinimler eklendiğinde, halihazırda test edilmiş ve çalışan mevcut kaynak kodun değiştirilmesini önlemektir. Mevcut kodu değiştirmek, sistemde beklenmedik hatalara (regression) yol açabilir. Bu prensip, genellikle arayüzler (interfaces) ve soyut sınıflar (abstract classes) kullanılarak uygulanan polimorfizm sayesinde gerçekleştirilir. Sisteme yeni bir davranış ekleneceği zaman, mevcut yapıya dokunulmadan yeni alt sınıflar türetilerek entegrasyon sağlanır.

```
// Base class for payment processing
abstract class PaymentProcessor {
    public abstract void processPayment(double amount); // Pure virtual function
}

// Credit card payment processor
class CreditCardPaymentProcessor extends PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// PayPal payment processor
class PayPalPaymentProcessor extends PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}

// Function to process payment
public static void processPayment(PaymentProcessor processor, double amount) {
    processor.processPayment(amount);
}

public class Main {
    public static void main(String[] args) {
        CreditCardPaymentProcessor creditCardProcessor = new CreditCardPaymentProcessor();
        PayPalPaymentProcessor paypalProcessor = new PayPalPaymentProcessor();

        processPayment(creditCardProcessor, 100.00); // Processing credit card payment
        processPayment(paypalProcessor, 150.00);    // Processing PayPal payment
    }

    // Function to process payment
    public static void processPayment(PaymentProcessor processor, double amount) {
        processor.processPayment(amount);
    }
}
```

Bu şekilde class extend edilerek bir üst classında herhangi bir değişiklik olmadan üzerine ekleme yapılmıştır.

3. Liskov Substitution Principle

Liskov Substitution prensibi, bir yazılım mimarisindeki temel (üst) sınıf nesnelerinin, programın doğruluğunu veya sistemin beklenen işleyişini bozmadan, kendi **alt sınıf nesneleriyle doğrudan değiştirilebilir** olması gerektiğini ifade eder.

Amacı ve İşlevi: Kalıtım (inheritance) hiyerarşisinin anlamsal ve yapısal olarak doğru kurulmasını sağlamaktır. Bir alt sınıf, miras aldığı üst sınıfın sözleşmelerine (contracts) ve beklenen davranışlarına *tamamen* sadık kalmalıdır. Alt sınıflar, üst sınıfların sunduğu metotları ezerken (override ederken) mevcut işleyişi bozan beklenmeyen kısıtlamalar getirmemelidir veya hata (exception) fırlatmamalıdır. Bu prensip ihlal edildiğinde, polimorfik olarak çalışan kod blokları öngörülemeyen hatalar üretir ve yazılımın bütünlüğü bozulur.

```
class Bird extends Animal {  
  fly() {  
    console.log(`${this.name} kanat çırpıyor`);  
  }  
}  
  
const parrot = new Bird('Titi the Parrot');  
makeAnimalSound(parrot); // Titi the Parrot bir ses çıkarıyor  
parrot.fly(); // Titi the Parrot kanat çırpıyor
```

4. Interface Segregation Principle

Arayüz Ayrımı Prensipli, istemcilerin (client sınıflarının) kullanmadıkları ve ihtiyaç duymadıkları metotlara bağımlı olmaya zorlanmamaları gerektiğini ifade eden yazılım tasarım kuralıdır. "Şişman" (fat) ve çok işlevli tek bir arayüz yerine, her biri belirli bir istemci grubuna veya amaca hizmet eden daha küçük, özelleştirilmiş ve yüksek kohezyona sahip arayüzler tasarlanmalıdır.

Amacı ve İşlevi: Sınıfları, yapamadıkları eylemler için metot gövdelerini boş bırakmaya veya `UnsupportedOperationException` gibi hatalar fırlatmaya zorlayan kötü tasarımların önüne geçmektir (ki bu durum aynı zamanda Liskov Prensipli'ni ve Single Responsibility Principle de ihlal eder). Arayüzlerin ayrıştırılması, sistemdeki bağımlılıkları (coupling) azaltır. Bir arayüzde meydana gelen yapısal bir değişiklik, **yalnızca o arayüzü gerçekten kullanan (implemente eden) sınıfları etkiler**, böylece kodun bakımı ve yeniden kullanılabilirliği kolaylaşır.

```
interface ShapeInterface
{
    public function area();

    public function volume();
}
```

Mesela böyle bir interface oluşturup kullanmamız 2D objeler için gereksiz olacaktır. 2D objelerin herhangi bir hacmi yoktur, dolayısı ile gereksiz bir arayüz implemente etmiş olurlar. Bu hatalı bir practice'dir. Bunun yerine:

```
interface ShapeInterface
{
    public function area();
}

interface ThreeDimensionalShapeInterface
{
    public function volume();
}
```

şeklinde 2D ve 3D objeler için gerekli olacak arayüzleri ikiye ayırıp buna göre inherit etmek daha sağlıklı bir practice'dir.

```

class Square implements ShapeInterface { // Only implements area()
    public $length;

    public function __construct($length) {
        $this->length = $length;
    }

    public function area() {
        return pow($this->length, 2);
    }
}

class Cuboid implements ShapeInterface, ThreeDimensionalShapeInterface
{
    public function area()
    {
        // calculate the surface area of the cuboid
    }

    public function volume()
    {
        // calculate the volume of the cuboid
    }
}

```

Bu sayede 2D olan kare objesinin hatalı şekilde hacim fonksiyonunu kalıtım olarak almış olmasından kurtuluruz.

5. Dependency Inversion Principle

Dependency Inversion prensibi, yüksek seviyeli modüllerin (sistemin ana iş mantığını yürüten sınıflar) düşük seviyeli modüllere (donanım, veritabanı, dosya sistemleri gibi detaylar) doğrudan bağımlı olmaması gerektiğini savunur. Bu bağımlılık, her iki tarafın da ortak bir soyutlama (abstraction) üzerinde birleşmesi ile tersine çevrilmelidir.

Amacı ve İşlevi: Sistemin modülleri arasındaki **sıkı bağımlılığı (tight coupling)** ortadan kaldırmaktır. Kodun içerisine doğrudan somut sınıf isimleri (new MySQLDatabase()) yazmak yerine, bu sınıfın arayüzüne (Database) bağımlı olmak, kodun gelecekte başka bir teknolojiye (örneğin PostgreSQL veya MongoDB) geçişini, ana iş mantığını değiştirmeden sağlar. Aynı zamanda Open-Closed prensiplerinin de ihlal edilmediği bir prensiptir. Bağımlılık ortadan kaldırılınca koddaki modifikasyon gerekliliği de kalkmış olur.

```
class MySQLConnection
{
    public function connect()
    {
        // handle the database connection
        return 'Database connection';
    }
}

class PasswordReminder
{
    private $dbConnection;

    public function __construct(MySQLConnection $dbConnection)
    {
        $this->dbConnection = $dbConnection;
    }
}
```

Kodda da gözlemlenebileceği üzere PasswordReminder sınıfı MySQLConnection sınıfına bağımlı olmuştur. **Tight coupling!** Burada MySQL sınıfı düşük seviye, PasswordReminder ise yüksek seviyedir. Bu bağımlılık Dependency Inversion Principle'i ihlal eder. Yüksek seviyeli modüller düşük seviyelilere bağımlı **olmamalıdır!** Bu bağımlılığı kaldırmak için yeni bir interface oluşturalım, PasswordReminder'i ona bağlayalım.

```
interface DBConnectionInterface
{
    public function connect();
}
```

```
class MySQLConnection implements DBConnectionInterface
{
    public function connect()
    {
        // handle the database connection
        return 'Database connection established';
    }
}
```

```
class PasswordReminder{
    private $dbConnection;

    public function __construct(DBConnectionInterface $dbConnection) // Typ
    {
        $this->dbConnection = $dbConnection;
    }

    public function remind() {
        $connectionStatus = $this->dbConnection->connect();
        return "Password reminder process initiated. Connection status: "
    }
}
```

```
// Create a concrete MySQL connection object
$mysqlConnector = new MySQLConnection();

// Inject the concrete MySQL connection object into the PasswordReminder
// The PasswordReminder only sees it as a DBConnectionInterface
$passwordReminder = new PasswordReminder($mysqlConnector);
```

```
$pgConnector = new PostgreSQLConnection();
$passwordReminder = new PasswordReminder($pgConnector);
```

Bu şekilde PasswordReminder'ı bir interface'e bağlarsak sonradan connectionumuzu değiştirmek tüm classları modifikasyondan geçirmektense tek satıra indirgenecektir.